

Système d'exploitation

Laurent Amanton



Université du Havre
UFR des Sciences et Techniques
bureau B 110

email : `laurent.amanton@univ-lehavre.fr`





Sommaire du chapitre

1 programmation en shell (bash)



À quoi ça sert ?

1^{er} processus d'une session

- Conteneur d'une session
- user: password: uid: gid: comment: home: **shell**



À quoi ça sert ?

1^{er} processus d'une session

- Conteneur d'une session
- user: password: uid: gid: comment: home: shell

interpréteur de commandes

- lecture et exécution de commandes
- redirection des entrées sorties
- expansion des noms de fichiers
- interface utilisateur (historique, complétion, alias, gestion de jobs...)



À quoi ça sert ?

1^{er} processus d'une session

- Conteneur d'une session
- user: password: uid: gid: comment: home: shell

interpréteur de commandes

- lecture et exécution de commandes
- redirection des entrées sorties
- expansion des noms de fichiers
- interface utilisateur (historique, complétion, alias, gestion de jobs...)

langage de programmation

- programmation interactive ou par fichiers de commandes
- instructions conditionnelles et itératives, fonctions
- gestion de variables
- substitution de paramètres



Différents Shells



Différents Shells

- sh ("Bourne shell", Steve Bourne)
shell disponible sur tous les Unix.
- csh ("C-shell", Bill Joy)
shell BSD disponible sur tous les Unix.
- ksh ("Korn shell")
shell normalisé (POSIX et ISO (HP, Solaris))
- zsh ("Zero shell")
shell du domaine public
- tcsh ("Toronto C-shell")



Différents Shells

- sh ("Bourne shell", Steve Bourne)
shell disponible sur tous les Unix.
- csh ("C-shell", Bill Joy)
shell BSD disponible sur tous les Unix.
- ksh ("Korn shell")
shell normalisé (POSIX et ISO (HP, Solaris))
- zsh ("Zero shell")
shell du domaine public
- tcsh ("Toronto C-shell")
- bash
installé sur toutes les machines unix et linux



Différents Shells

- sh ("Bourne shell", Steve Bourne)
shell disponible sur tous les Unix.
- csh ("C-shell", Bill Joy)
shell BSD disponible sur tous les Unix.
- ksh ("Korn shell")
shell normalisé (POSIX et ISO (HP, Solaris))
- zsh ("Zero shell")
shell du domaine public
- tcsh ("Toronto C-shell")
- bash ("[Bourne again shell](#)" de Brian Fox et Chet Ramey)
installé sur toutes les machines unix et linux



un mini OS ...

Ouverture des fichiers standards

entrée standard (stdin, fd=0)

sortie standard (stdout, fd=1)

sortie erreur standard (stderr, fd=2)



un mini OS ...

Ouverture des fichiers standards

entrée standard (stdin, fd=0)

sortie standard (stdout, fd=1)

sortie erreur standard (stderr, fd=2)

Le shell lit sur son entrée standard

lecture de la commande jusqu'à la fin de ligne (interprétation des métacaractères et opérateurs)

lecture des lignes de commandes jusqu'à la fin de fichier



un mini OS ...

Ouverture des fichiers standards

entrée standard (stdin, fd=0)

sortie standard (stdout, fd=1)

sortie erreur standard (stderr, fd=2)

Le shell lit sur son entrée standard

lecture de la commande jusqu'à la fin de ligne (interprétation des métacaractères et opérateurs)

lecture des lignes de commandes jusqu'à la fin de fichier

Lancement d'une commande

recherche du nom de la commande à l'aide du path

en premier plan : fork() exec(commande) wait() prompt

en arrière plan : fork() exec(commande) prompt



Caractères de contrôle (clavier)

[^]C



Caractères de contrôle (clavier)

[^]C interruption d'un processus attaché au terminal

[^]Z



Caractères de contrôle (clavier)

- ^C** interruption d'un processus attaché au terminal
- ^Z** suspension d'un processus en premier plan



Caractères de contrôle (clavier)

- ^C** interruption d'un processus attaché au terminal
- ^Z** suspension d'un processus en premier plan
- ^\
^_** arrêt d'un processus avec vidage mémoire (core)
- ^U** effacement de la ligne complète
- ^D** fin de fichier (si le shell lit, fin du shell)
- ^S** suspension de l'affichage écran (Xoff)
- ^Q** reprise de l'affichage écran (Xon)



Enchaînement de commandes

- Commande simple : `com [arg]`



Enchaînement de commandes

- Commande simple : `com [arg]`
- Commande simple en arrière plan : `com [arg] &`



Enchaînement de commandes

- Commande simple : `com [arg]`
- Commande simple en arrière plan : `com [arg] &`
- Pipeline (tube de communication) : `com1 [arg] | com2 [arg]`



Enchaînement de commandes

- Commande simple : `com [arg]`
- Commande simple en arrière plan : `com [arg] &`
- Pipeline (tube de communication) : `com1 [arg] | com2 [arg]`
- Séquence de commandes : `com1 [arg] ; com2 [arg]`



Enchaînement de commandes

- Commande simple : `com [arg]`
- Commande simple en arrière plan : `com [arg] &`
- Pipeline (tube de communication) : `com1 [arg] | com2 [arg]`
- Séquence de commandes : `com1 [arg] ; com2 [arg]`
- Exécution dans un sous-shell :



Enchaînement de commandes

- Commande simple : `com [arg]`
- Commande simple en arrière plan : `com [arg] &`
- Pipeline (tube de communication) : `com1 [arg] | com2 [arg]`
- Séquence de commandes : `com1 [arg] ; com2 [arg]`
- Exécution dans un sous-shell :
 - `sh com [arg]`



Enchaînement de commandes

- Commande simple : `com [arg]`
- Commande simple en arrière plan : `com [arg] &`
- Pipeline (tube de communication) : `com1 [arg] | com2 [arg]`
- Séquence de commandes : `com1 [arg] ; com2 [arg]`
- Exécution dans un sous-shell :
 - `sh com [arg]`
 - `(com [arg])` ou `(com [arg])&`



Enchaînement de commandes

- Commande simple : `com [arg]`
- Commande simple en arrière plan : `com [arg] &`
- Pipeline (tube de communication) : `com1 [arg] | com2 [arg]`
- Séquence de commandes : `com1 [arg] ; com2 [arg]`
- Exécution dans un sous-shell :
 - `sh com [arg]`
 - `(com [arg])` ou `(com [arg])&`
- Regroupement de commandes :



Enchaînement de commandes

- Commande simple : `com [arg]`
- Commande simple en arrière plan : `com [arg] &`
- Pipeline (tube de communication) : `com1 [arg] | com2 [arg]`
- Séquence de commandes : `com1 [arg] ; com2 [arg]`
- Exécution dans un sous-shell :
 - `sh com [arg]`
 - `(com [arg])` ou `(com [arg])&`
- Regroupement de commandes :
 - `(com1 [arg] ; com2 [arg])`



Enchaînement de commandes

- Commande simple : `com [arg]`
- Commande simple en arrière plan : `com [arg] &`
- Pipeline (tube de communication) : `com1 [arg] | com2 [arg]`
- Séquence de commandes : `com1 [arg] ; com2 [arg]`
- Exécution dans un sous-shell :
 - `sh com [arg]`
 - `(com [arg])` ou `(com [arg])&`
- Regroupement de commandes :
 - `(com1 [arg] ; com2 [arg])`
 - `(com1 [arg] ; com2 [arg]) &`



Redirections

redirection de l'entrée standard

< fichier ou périphérique



Redirections

redirection de l'entrée standard

< fichier ou périphérique

<< *balise* texte jusqu'à la balise (*here doc*)

redirection de la sortie standard

> fichier ou périphérique

1 > fichier ou périphérique



Redirections

redirection de l'entrée standard

< fichier ou périphérique

<< *balise* texte jusqu'à la balise (*here doc*)

redirection de la sortie standard

> fichier ou périphérique

1 > fichier ou périphérique

>> ajout (en fin de fichier)



Redirections

redirection de la sortie erreur

- > & fichier ou périphérique
- & > fichier ou périphérique
- 2 > fichier ou périphérique



Exemples de redirections

- ❶ `cat f1 > f2`
- ❷ `cat f1 >> f2`
- ❸ `cat > f`
- ❹ `ls > f`
- ❺ `cat < f`
- ❻ `cat < f1 > f2`
- ❼ `cat > f2 < f1`
- ❽ `cat < f1 >> f2`
- ❾ `commande 1 >/dev/null 2 > &1/dev/null`
- ❿ `rm *.bak > & f`



Caractères d'expansion (des noms de fichiers)

- $[xyz]$ coïncidence avec x , y ou z
 - $*$ remplace n'importe quoi (y compris rien)
 - $?$ remplace un et un seul caractère
- $[x - z]$ coïncidence dans l'intervalle $[x, y]$
- $\{x, yyy\}$ remplace successivement les chaînes x et yyy
- $\sim$ remplace par `HOME`



Caractères d'expansion (des noms de fichiers)

Exemples

❶ `ls *a.*`

❷ `ls [A-Z]*.[mp3-6]`

❸ `ls *.{ps,pdf}`

❹ `cd ~`



Variables d'environnement (env)

PATH suite de chemins

MAIL chemin d'accès à la boîte mail

ENV fichier de toutes les variables d'environnement

TERM nom du type de terminal

LOGNAME nom de login

USER nom de l'utilisateur actuel

...



Variables de substitution

\$0	nom du script (pathname)
\$1, ..., \$9	arguments
\$#	nombre d'arguments
\$*	liste de tous les arguments
\$@	liste de tous les arguments
\$_	code de retour de la dernière commande
\$\$	numéro de processus de ce shell
\$_	numéro du dernier processus en arrière plan



Variables utilisateur

Déclaration

`variable=valeur`

Utilisation

`$variable` ou `${variable}`

Exportation

`export variable=valeur (bash)`



Variables utilisateur

Déclaration

`variable=valeur`

Utilisation

`$variable` ou `${variable}`

Exportation

`export variable=valeur (bash)`

exemple

`toto='bla bla bla'`



Condition

condition simple

```
if commande ; then  
    liste de commandes  
fi
```



Condition

condition simple

```
if commande ; then  
    liste de commandes  
fi
```

condition et alternative

```
if commande ; then  
    liste de commandes  
else  
    liste de commandes  
fi
```



Condition

conditions multiples

```
if commande ; then
    liste de commandes
elif commande ; then
    liste de commandes
else
    liste de commandes
fi
```




Conditions multiples par aiguillage

Aiguillage

```
case $variable in
    motif 1) liste de commandes ;;
    motif 2) liste de commandes ;;
    *) liste de commandes ;;
esac
```



Conditions multiples par aiguillage

Aiguillage

```
case $variable in
    motif 1) liste de commandes ;;
    motif 2) liste de commandes ;;
    *) liste de commandes ;;
esac
```

exemple

```
case $# in
    1) a=$1;;
    2) a=$1; b=$2 ;;
    0) echo "usage:  $0 a [ b ]" ; exit 1;;
esac
```



Boucles

for

```
for variable [ in liste ]
```

```
do
```

```
    liste de commandes
```

```
done
```



Boucles

for

```
for variable [ in liste ]
```

```
do
```

```
    liste de commandes
```

```
done
```

while

```
while commande
```

```
do
```

```
    liste de commandes
```

```
done
```



Boucles

for

```
for variable [ in liste ]
```

```
do
```

```
    liste de commandes
```

```
done
```

while

```
while commande
```

```
do
```

```
    liste de commandes
```

```
done
```

until

```
until commande
```

```
do
```

```
    liste de commandes
```

```
done
```



Boucles

exemple

```
for f in *.bak ; do
    rm $f
done
```

```
verif="n"
while [ "$verif" != o ]
do
    echo "Entrer une valeur : "
    read valeur
    echo "vous avez entré $valeur. Correct ? (o/n)"
    read verif
done
```



Rupture de boucles

Sortie de boucle

`break [n]`

n est le niveau de boucle depuis la boucle la plus externe



Rupture de boucles

Sortie de boucle

`break [n]`

n est le niveau de boucle depuis la boucle la plus externe

Reprise de boucle

`continue [n]`



expansion de paramètres

<code>\${var:-val}</code>	si var définie ou non nulle alors var sinon val
<code>\${var:?mess}</code>	si var définie ou non nulle alors var sinon affiche mess
<code>\${var:+val}</code>	si var définie ou non nulle alors val sinon rien
<code>\${#var}</code>	longueur en octet de la chaîne var
<code>\${var:début:long}</code>	sous-chaîne
<code>\${var%str}</code>	supprime en fin de var la plus petite chaîne des 2



expansion de paramètres

<code>\${var:-val}</code>	si var définie ou non nulle alors var sinon val
<code>\${var:?mess}</code>	si var définie ou non nulle alors var sinon affiche mess
<code>\${var:+val}</code>	si var définie ou non nulle alors val sinon rien
<code>\${#var}</code>	longueur en octet de la chaîne var
<code>\${var:début:long}</code>	sous-chaîne
<code>\${var%str}</code>	supprime en fin de var la plus petite chaîne des 2

renommage

```
for f in *.mp3 ; do
    fiche=${f%.mp3} ; mv $f $fiche.ogg
done
```



Expansion de commandes

expansion

- variable='commande '
- variable=\${commande}



Expansion de commandes

expansion

- `variable='commande '`
- `variable=$(commande)`

exemples

- `date='date +%d/%m/%y'`
- `dir='dirname $fichier '`
- `nomfichier=$(basename $fichier )`



Quotes

quotes simples ' pas d'évaluation des méta-caractères

quotes doubles " expansion pour \$ ' et \



Quotes

quotes simples ' pas d'évaluation des méta-caractères

quotes doubles " expansion pour \$ ' et \

exemple

```
echo "usage: 'basename $0' '$HOME'"
```



Interruptions

mise en place d'un déroutement

trap 'liste de commandes' liste de signaux

exemple : `trap 'rm /tmp/*' ; exit 0 2 3`



Interruptions

mise en place d'un déroutement

trap 'liste de commandes' liste de signaux

exemple : `trap 'rm /tmp/*' ; exit 0 2 3`

désactivation des signaux

trap " liste de signaux

exemple : `trap " 1 2 18`



Interruptions

mise en place d'un déroutement

trap 'liste de commandes' liste de signaux

exemple : `trap 'rm /tmp/*' ; exit 0 2 3`

désactivation des signaux

trap " liste de signaux

exemple : `trap " 1 2 18`

Repositionnement par défaut des signaux

trap liste de signaux

exemple : `trap 1 2 3`



Interruptions

mise en place d'un déroutement

trap 'liste de commandes' liste de signaux

exemple : `trap 'rm /tmp/*' ; exit 0 2 3`

désactivation des signaux

trap " liste de signaux

exemple : `trap " 1 2 18`

Repositionnement par défaut des signaux

trap liste de signaux

exemple : `trap 1 2 3`

Liste des signaux déroutés

trap



Test

test sur les fichiers

`test -option fichier`



Test

test sur les fichiers

test -option fichier

- f fichier ?
- d répertoire ?
- s fichier de taille non nulle ?
- r en lecture ?
- w en écriture ?
- x exécutable ?
- e existe ?
- ...



Test

test sur les fichiers

test -option fichier

- f** fichier ?
- d** répertoire ?
- s** fichier de taille non nulle ?
- r** en lecture ?
- w** en écriture ?
- x** exécutable ?
- e** existe ?

...

exemple

```
if [ ! -d repertoire ]; then mkdir repertoire ; fi
```



Test

tests algébriques

test nombre1 -option nombre2



Test

tests algébriques

test nombre1 **-option** nombre2

-eq equal ?

-ne non equal ?

-gt greater than ?

-lt less than ?

-ge greater or equal ?

-le less or equal ?



Test

tests algébriques

test nombre1 **-option** nombre2

-eq equal ?

-ne non equal ?

-gt greater than ?

-lt less than ?

-ge greater or equal ?

-le less or equal ?

exemple

```
if [ $a -eq 2 ]; then a=0 ; fi
```




Test

comparaisons alphanumériques

test chaîne1 option chaîne2



Test

comparaisons alphanumériques

test chaîne1 **option** chaîne2

= égal ?

!= différent ?

> supérieur strict ?

< inférieur strict ?

>= supérieur ou égal ?

<= inférieur ou égal ?



Test

comparaisons alphanumériques

test chaîne1 **option** chaîne2

= égal ?

!= différent ?

> supérieur strict ?

< inférieur strict ?

>= supérieur ou égal ?

<= inférieur ou égal ?

exemple

```
if [ $chaine = "toto" ]; then ... ; fi
```



Test

comparaisons booléennes

`test -z chaîne` chaîne de longueur zéro ?

`test -n chaîne` chaîne de longueur non nulle ?



Test

comparaisons booléennes

`test -z chaîne` chaîne de longueur zéro ?

`test -n chaîne` chaîne de longueur non nulle ?

divers

`!` not

`-a` et logique (and)

`-o` ou logique (or)

`-nt` fichier plus récent qu'un autre

`-ot` fichier plus ancien qu'un autre



Arithmétique

Évaluation d'expressions

expr valeur1 **opérateur** valeur2

opérateurs $+$ $-$ $*$ $/$ $\%$

pas très performant



Arithmétique

Évaluation d'expressions

expr valeur1 **opérateur** valeur2

opérateurs + - * / %

pas très performant

exemple

```
compteur='expr $compteur + 1'
```



Arithmétique

Évaluation d'expressions

`$[ valeur1 opérateur valeur2 ]`

opérateurs `+` `-` `*` `/` `%`

très performant mais dépendant du shell
(inexistant dans sh)



Arithmétique

Évaluation d'expressions

`$[ valeur1 opérateur valeur2 ]`

opérateurs `+` `-` `*` `/` `%`

très performant mais dépendant du shell
(inexistant dans sh)

exemple

```
compteur= $[ $compteur + 1 ]
```



Fonctions internes

alias [nom[=commande] ...]	définition d'une commande
unalias [nom ...]	suppression d'un alias
bg [jobnum]	background
fg [jobnum]	foreground
echo [-ne] [arg ...]	affichage
eval [arg ...]	évaluation d'une expression
exec [[-] commande[arg ...]]	exécution d'une commande
exit [n]	sortie du script
kill [-signal] [pid jobnum]	envoi de signaux
read [variable ...]	lecture au clavier
return [n]	retour d'une fonction



Fonctions internes

set [-option] [arg ...]	positionne des variables
unset [variable ...]	supprime une variable
shift [n]	décalage des paramètres \$i
test expression	condition
times	données temporelles
trap [arg] [signaux]	déroutements
ulimit [-option [limite]]	limitation des ressources
umask [mode]	masque utilisateur (chmod)
wait [pid jobnum]	synchronisation



Fonctions utilisateur

Deux déclarations possibles

- `function nom_fonction { ... }`
- `nom_fonction () { ... }`



Fonctions utilisateur

Deux déclarations possibles

- **function** *nom_fonction* { ... }
- *nom_fonction* () { ... }

exemple

```
main()
{
    if [ $# = 0 ]; then
        echo "1 paramètre !"
        exit 1
    fi
}
```



Fonctions utilisateur

- Les fonctions doivent être définies avant leur appel
- Aucun argument lors de la déclaration d'une fonction
- Appel d'une fonction : *nom_fonction* *arg1 arg2 ...*
- Récupération des arguments : \$1 \$2 ...



Fonctions utilisateur

- Les fonctions doivent être définies avant leur appel
- Aucun argument lors de la déclaration d'une fonction
- Appel d'une fonction : *nom_fonction* *arg1 arg2 ...*
- Récupération des arguments : \$1 \$2 ...

exemple

```
usage ()  
{  
    echo "usage:  $1 $2"  
}  
  
usage `basename $0` "nom_de_fichier"
```



Script shell

Définition

?



Script shell

Définition

ensemble de commandes shell directement interprétables...



Script shell

Écriture d'un script

`#!/bin/bash`

`#` localisation de l'interpréteur

Déclaration des variables

Déclaration des fonctions

appel de la fonction principale

exemple

```
#!/bin/bash

usage()
{
    echo "usage:  $1 $2"
}

function main
{
    if [ $# = 0 ]; then
        usage `basename $0` "nom_de_fichier"
        exit 1
    fi
}

main $*
```



Les scripts shell

Questions

